

4.6 — Design Multi-Instance and Multi-Pass Review Architectures

Concept

When you ask a model to review its own output, you are asking it to evaluate something it already agreed with. The model produced the original content under a particular reasoning path, with particular biases and blind spots. Asking the same instance to re-examine that content in the same conversation tends to produce confirmatory rather than adversarial review. The model is not lazy — it is coherent. Coherence is the problem.

Self-review is structurally unreliable for three reasons:

1. **Shared context bias.** The reviewer has access to all prior reasoning. It can reconstruct why each decision was made and will typically treat that reasoning as justification rather than as evidence to challenge.
2. **Conversation-level anchoring.** Earlier messages in a conversation exert pull on later ones. A model that wrote a function is primed to interpret that function charitably.
3. **No independent prior.** A truly independent reviewer comes in without preconceptions. A self-reviewer cannot manufacture ignorance of what it just wrote.

The architectural response is to separate authorship from review at the API call level. Independent review instances are distinct `messages.create` calls with no shared conversation history and no access to the original reasoning context. They receive only the artifact — the code, the document, the plan — not the process that produced it.

Multi-Pass Architecture

Large codebases or complex outputs benefit from a staged review pipeline rather than a single omnibus review:

1. **Per-file analysis pass.** Each file is reviewed in isolation by a dedicated instance. The scope is narrow: this file, against this set of criteria. Findings are structured and stored.
2. **Cross-file integration pass.** A second layer of instances receives pairs or clusters of files plus the per-file findings. Its job is to identify issues that only become visible when two components are seen together: interface mismatches, inconsistent error handling conventions, shared state mutations.
3. **Final synthesis pass.** A single aggregating instance receives all findings from both prior passes, deduplicates, assigns priorities, and produces an actionable summary. It does not re-read the source files — it works from findings only.

This staged structure keeps individual context windows small, parallelizes naturally, and ensures that no single model instance is responsible for both generating and validating a conclusion.

Split-Role Reviewers

Assigning distinct roles to distinct instances improves precision and reduces false positives from mixed concerns. Three roles cover most review scenarios:

- **Security reviewer** — checks for injection, authentication bypass, secret leakage, unsafe deserialization. Its system prompt includes only security criteria.
- **Performance reviewer** — checks for N+1 query patterns, unbounded loops, synchronous I/O on the hot path, unnecessary re-renders. Its system prompt includes only performance criteria.
- **Correctness reviewer** — checks for logic errors, unhandled edge cases, incorrect type assumptions, missing error propagation. Its system prompt includes only correctness criteria.

These three instances run in parallel on the same file. Their findings are structurally tagged by role, so the aggregation pass can route critical security findings to a security team without wading through performance observations.

Code Example

Independent review instances with split roles — parallel per-file pass

```
import Anthropic from "@anthropic-ai/sdk";

const client = new Anthropic();

type ReviewRole = "security" | "performance" | "correctness";

interface Finding {
  role: ReviewRole;
  file: string;
  line: number | null;
  severity: "critical" | "high" | "medium" | "low";
  description: string;
  remediation: string;
}

interface PassResult {
  file: string;
  role: ReviewRole;
  findings: Finding[];
}

const ROLE_PROMPTS: Record<ReviewRole, string> = {
  security: `You are a security code reviewer. Check only for:
- SQL injection (user input concatenated into queries without parameterization)
- XSS (user input rendered into HTML without sanitization)
- Authentication bypass (protected routes reachable without a valid session)
- Hardcoded secrets (API keys or passwords as string literals)
Do not report performance issues, style issues, or correctness bugs.`,
```

```
performance: `You are a performance code reviewer. Check only for:
- N+1 query patterns (database calls inside loops)
- Unbounded loops iterating over unsized collections
- Synchronous I/O blocking the event loop
- Missing memoization on expensive pure computations called repeatedly
Do not report security issues, style issues, or correctness bugs.`,
```

```
correctness: `You are a correctness code reviewer. Check only for:
- Unhandled promise rejections or missing error propagation
- Off-by-one errors in array indexing or loop bounds
- Incorrect null/undefined assumptions (accessing properties before checking)
- Type coercions that silently produce NaN or wrong values
Do not report security issues, performance issues, or style suggestions.`,
```

```
};

async function reviewFileWithRole(
  file: string,
  code: string,
  role: ReviewRole
): Promise<PassResult> {
  // Each call is a fresh API request – no shared conversation history
  const response = await client.messages.create({
    model: "claude-sonnet-4-5",
    max_tokens: 2048,
    system: ROLE_PROMPTS[role],
    messages: [
      {
        role: "user",
        content: `Review the following file and return findings as JSON.`
```

Schema:

```
{
  "findings": [
    {
      "role": "${role}",
      "file": "${file}",
      "line": <number | null>,
      "severity": "<critical|high|medium|low>",
      "description": "<what you found>",
      "remediation": "<specific fix>"
    }
  ]
}
```

If you find nothing, return an empty findings array. Do not add commentary outside the JSON block.

```

\`\`\`typescript
${code}
\`\`\`,
    },
    ],
  });

const block = response.content[0];
if (block.type !== "text") throw new Error("Unexpected response type");

const match = block.text.match(/\{[\s\S]*\}/);
if (!match) throw new Error(`No JSON in ${role} response for ${file}`);

const parsed = JSON.parse(match[0]) as { findings: Finding[] };
return { file, role, findings: parsed.findings };
}

async function perFilePass(
  files: Record<string, string>
): Promise<PassResult[]> {
  const roles: ReviewRole[] = ["security", "performance", "correctness"];

  // All role × file combinations run in parallel – no shared state
  const tasks = Object.entries(files).flatMap(([file, code]) =>
    roles.map((role) => reviewFileWithRole(file, code, role))
  );

  return Promise.all(tasks);
}

```

Cross-file integration pass — finds issues invisible in isolation

```

async function crossFilePass(
  filePairs: Array<{ fileA: string; codeA: string; fileB: string; codeB: string }>,
  priorFindings: PassResult[]
): Promise<Finding[]> {
  const tasks = filePairs.map(async ({ fileA, codeA, fileB, codeB }) => {
    const relevantFindings = priorFindings
      .filter((r) => r.file === fileA || r.file === fileB)
      .flatMap((r) => r.findings);

    const response = await client.messages.create({
      model: "claude-sonnet-4-5",
      max_tokens: 2048,
      system:
        "You are an integration reviewer. Your job is to find issues that only appear when two

```

```

    messages: [
      {
        role: "user",
        content: `Prior findings for these files:
${JSON.stringify(relevantFindings, null, 2)}

File A (${fileA}):
\\\\"\\\\"typescript
${codeA}
\\\\"\\\\"

File B (${fileB}):
\\\\"\\\\"typescript
${codeB}
\\\\"\\\\"

Return only new integration-level findings as JSON using the same Finding schema. Return an empty array if no new findings are found.

    },
    ],
  });

  const block = response.content[0];
  if (block.type !== "text") return [];

  const match = block.text.match(/\\{\\[\\s\\S]*\\}/);
  if (!match) return [];

  const parsed = JSON.parse(match[0]) as { findings: Finding[] };
  return parsed.findings;
});

const results = await Promise.all(tasks);
return results.flat();
}

```

Final synthesis pass — aggregate and prioritize

```

async function synthesisPass(allFindings: Finding[]): Promise<string> {
  // This instance never reads source files – it works from findings only
  const response = await client.messages.create({
    model: "claude-sonnet-4-5",
    max_tokens: 2048,
    system:
      "You are a review coordinator. Deduplicate findings, group by severity, and produce a prioritized list of findings for the developer to address.",
    messages: [
      {

```

```
        role: "user",
        content: `Aggregate these findings into a prioritized summary. Remove duplicates. Group
    },
    ],
});

const block = response.content[0];
return block.type === "text" ? block.text : "";
}
```

Common Mistakes

- 1. Passing the original conversation history to the reviewer.** If the review instance can see the messages that produced the artifact, it inherits the same anchoring bias as a self-review. Always start review calls with a clean `messages` array.
- 2. Running per-file and cross-file passes in the same instance.** A single large-context review of many files asks one model to hold all files simultaneously. This degrades attention on individual files and defeats the purpose of per-file scoping. Keep passes separate.
- 3. Skipping the synthesis pass and acting on raw findings.** Raw findings from multiple parallel instances will contain duplicates and conflicting severities. Without aggregation, automated pipelines double-count issues and generate redundant alerts.
- 4. Giving all roles the same system prompt.** If security, performance, and correctness reviewers share a prompt, each instance will attempt to cover all concerns, producing redundant overlap and defeating role isolation. Each role needs an exclusive scope definition, including explicit instructions to ignore the other categories.
- 5. Using the synthesis instance to re-read source files.** The synthesis pass exists to operate on findings, not code. Loading source files into the synthesis context inflates token usage and re-introduces the risk of the synthesizer overriding earlier specialist findings with a less-focused general impression.

References

- Anthropic Docs — [Building with the Messages API](#)
- Anthropic Cookbook — [Multi-agent orchestration patterns](#)
- Domain mental model: *Independent instances beat self-review. Narrow scope beats omnibus review. Findings beat re-reading.*